

Can dbt transform parquet files stored in S3?

Yes. dbt compiles SQL and hands it to a query engine; the engine reads the parquet files in place over S3. Nothing is copied or loaded into a warehouse first. Proven live on two engines with identical models and identical results; a third (Databricks) is built and parse-validated.

Question raised by Andy Ellman. Tested live 2026-08-18 by Jason Chletsos, Fivetran SE. All numbers below are from actual runs.

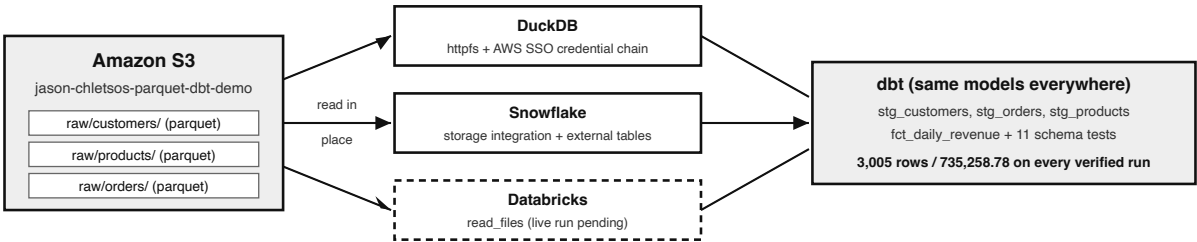
TEST SETUP

Sample trading-card storefront data: 500 customers, 50 products, 5,000 orders across 5 parquet files in `s3://jason-chletsos-parquet-dbt-demo/raw/{customers,products,orders}/` (us-east-1, Fivetran internal sales AWS account), generated by `data_generator/generate_parquet.py`. The same dbt models are defined for every engine: three staging views (`stg_customers`, `stg_orders`, `stg_products`), one table (`fct_daily_revenue`), and 11 schema tests. 15 build steps per run.

MECHANISM PER ENGINE

Engine	How it reads S3 parquet	Result
DuckDB (local files)	Baseline: same models over local copies of the parquet files.	15 of 15 steps pass.
DuckDB (S3 direct)	<code>httpfs</code> extension reads <code>s3://</code> paths directly; auth via the AWS SSO credential chain.	15 of 15 steps pass.
Snowflake	Storage integration (IAM role trust handshake) plus external tables declared with the <code>dbt_external_tables</code> package. Account: <code>sales_eng_demo</code> .	15 of 15 steps pass.
Databricks	<code>read_files</code> over <code>s3://</code> . Project built and parse-validated; IAM role provisioned (<code>role/jason-chletsos-parquet-demo-databricks</code>).	Live run pending a workspace and Unity Catalog external location.

ARCHITECTURE



RESULT PARITY

Every completed run produced the same `fct_daily_revenue` output: **3,005 rows**, gross revenue **735,258.78**. DuckDB local, DuckDB over S3, and Snowflake external tables all agree to the cent. That parity is the point: the transformation layer is portable because dbt owns the SQL and the engine is swappable.

TWO GOTCHAS HIT AND FIXED

- Cross-cloud external tables need `auto_refresh = false`**
The `sales_eng_demo` Snowflake account is not hosted on AWS. Snowflake defaults external-table auto refresh to true and errors on cross-cloud S3 stages. Setting `auto_refresh = false` explicitly in the external-table definition resolves it.
- TIMESTAMP_TZ silently shifts UTC instants**
Typing the parquet UTC timestamp as `TIMESTAMP_TZ` re-stamped the UTC wall-clock with the session timezone (America/Los_Angeles), shifting instants by 7 to 8 hours and skewing daily grain. Fix: `TIMESTAMP_NTZ` (naive UTC) plus a plain date cast; daily grain then matches across engines.

FIVETRAN ANGLE

This demo hand-uploads parquet; in production, Fivetran's Managed Data Lake Service does the landing. MDLS writes connector data to S3 as parquet-backed Iceberg tables that Fivetran maintains (compaction, schema evolution, catalog registration). The dbt-transforms-in-place pattern shown here is identical; Fivetran replaces the manual upload step. Details in `fivetran/INGESTION.md`.

INFRASTRUCTURE

Terraform provisions the S3 bucket and IAM roles for Snowflake and Databricks. The Snowflake trust handshake is the standard two-step: create the storage integration, then `DESC INTEGRATION` and stamp Snowflake's IAM user and external ID into the role trust policy.